

UNIVERSITY OF OXFORD
SOFTWARE ENGINEERING PROGRAMME

Wolfson Building, Parks Road, Oxford OX1 3QD, UK
Tel +44(0)1865 283525 Fax +44(0)1865 283531
info@softeng.ox.ac.uk www.softeng.ox.ac.uk

Part-time postgraduate study in software engineering



Quantum Computing, QUC

7th – 11th April 2025

ASSIGNMENT

The purpose of this assignment is to test the extent to which you have achieved the learning objectives of the course. As such, your answer must be substantially your own original work. Where material has been quoted, reproduced, or co-authored, you should take care to identify the extent of that material, and the source or co-author.

Your answers to the questions on this assignment should be submitted using the Software Engineering Programme website — www.softeng.ox.ac.uk — following the submission guidelines. When submitting the assignment online, it is important that you formally complete all three assignment submission steps: step 1, read through the declaration; step 2, upload your files; step 3, check your files. **Please ensure your submission is anonymous: do not include your name or any other identifying information on the assignment, nor in accompanying material such as source code, nor within the file names of anything submitted.**

The deadline for submission is 12 noon on Tuesday, 27th May 2025. You are strongly encouraged to submit a version well before the deadline. You may update your submission as often as you like before the deadline, but no submissions or changes will be accepted after the deadline.

We hope to have preliminary results and comments available during the week commencing Monday, 7th July 2025. The final results and comments will be available after the subsequent examiners' meeting. Exam Conventions can be found here <https://www.softeng.ox.ac.uk/handbook/>

**ANY QUERIES OR REQUESTS FOR CLARIFICATION
REGARDING THIS ASSIGNMENT OR PROBLEMS INSTALLING
SOFTWARE SHOULD, IN THE FIRST INSTANCE, BE DIRECTED
TO THE PROGRAMME OFFICE WITHIN THE NEXT TWO
WEEKS.**

QUC Assignment – Apr 2025

Tasks

In this assignment, you'll use problem Hamiltonians and QAOA to solve small instances of integer arithmetic problems.

Task 1

Implement problem Hamiltonians for the following constraints, to be imposed on selected bits b_i, b_j, b_k of a bitstring $\underline{b}$:

1. $b_k = b_j$ and b_i
2. $b_k = b_j$ or b_i
3. $b_k = b_j$ xor b_i
4. $b_j = \text{not } b_i$
5. $b_i = 0$
6. $b_i = 1$

Each constraint corresponds to a cost function which takes the value 1 if the constraint is violated on a bitstring $\underline{b}$, and the value 0 if the constraint is satisfied. For example, the first constraint corresponds to the following cost function:

$$f_{ijk}^{\text{and}}(\underline{b}) = \begin{cases} 0 & \text{if } b_k = b_i \text{ and } b_j \\ 1 & \text{if } b_k \neq b_i \text{ and } b_j \end{cases}$$

The first three problem Hamiltonians are 3-local, taking the following form for some coefficients:

$$c_{ijk}Z_iZ_jZ_k + c_{ij}Z_iZ_j + c_{ik}Z_iZ_k + c_{jk}Z_jZ_k + c_iZ_i + c_jZ_j + c_kZ_k + c$$

The fourth problem Hamiltonian is 2-local, taking the following form for some coefficients:

$$c_{ij}Z_iZ_j + c_iZ_i + c_jZ_j + c$$

The last two problem Hamiltonians are 1-local, taking the following form for some coefficients:

$$c_iZ_i + c$$

You should define functions as follows:

```
def and_hamiltonian(i, j, k, num_qubits) -> SparsePauliOp:
    ...

def or_hamiltonian(i, j, k, num_qubits) -> SparsePauliOp:
    ...

def xor_hamiltonian(i, j, k, num_qubits) -> SparsePauliOp:
    ...
```

```
def not_hamiltonian(i, j, num_qubits) -> SparsePauliOp:
    ...

def const0_hamiltonian(i, num_qubits) -> SparsePauliOp:
    ...

def const1_hamiltonian(i, num_qubits) -> SparsePauliOp:
    ...
```

You can work out the Paulis and coefficients for each individual Hamiltonian by using the `cost_to_problem_hamiltonian` function, but in your implementation of the six functions above you will have to use the given `i`, `j` and/or `k` as qubit indices when calling `SparsePauliOp.from_sparse_list`.

Task 2

Using the Hamiltonians from Task 1, implement the problem Hamiltonian for the half adder circuit (cf. [https://en.wikipedia.org/wiki/Adder_\(electronics\)#Half_adder](https://en.wikipedia.org/wiki/Adder_(electronics)#Half_adder)):

$$b_l, b_k = \text{half-adder}(b_j, b_i)$$

where b_i and b_j are variables corresponding to the two inputs of the half adder, b_k corresponds to the carry output, and b_l corresponds to the sum output. Specifically, this corresponds to the following constraints:

$$\begin{aligned} b_k &= b_j \text{ and } b_i \\ b_l &= b_j \text{ xor } b_i \end{aligned}$$

Your implementation should take the form of a function:

```
def half_adder_hamiltonian(i, j, k, l, num_qubits) -> SparsePauliOp:
    ...
```

You can check that your implementation of the problem Hamiltonian is correct by iterating over all 4-bitstrings (use `bitstrings(4)`) and using `eval_problem_hamiltonian` to check that you get the correct value on each bitstring.

Modify the Hamiltonian, using the constant value Hamiltonians from Task 1, to create the problem Hamiltonian for the constraint $0,1 = \text{half-adder}(b_j, b_i)$. Use QAOA (multiple layers, if necessary) to find solutions b_j, b_i to this problem.

Task 3

Using the Hamiltonians from Task 1, implement the problem Hamiltonian for the full adder circuit (cf. [https://en.wikipedia.org/wiki/Adder_\(electronics\)#Full_adder](https://en.wikipedia.org/wiki/Adder_(electronics)#Full_adder)):

$$b_m, b_l = \text{full-adder}(b_k, b_j, b_i)$$

where b_i and b_j are variables corresponding to the two input summands of the full adder, b_k is the variable corresponding to the input carry, b_l corresponds to the carry output, and b_m corresponds to the sum output.

Note that the full constraint involves 3 auxiliary variables b_{x_0} , b_{x_1} and b_{x_2} , used to store intermediate results, for a total of 8 variables. Your implementation should take the form of a function:

```
def full_adder_hamiltonian(i, j, k, l, m, x0, x1, x2, num_qubits) -> SparsePauliOp:
    ...
```

Modify the Hamiltonian and use QAOA (multiple layers, if necessary) to compute $b_m, b_l = \text{full-adder}(b_k, b_j, b_i)$ for all each fixed triple of values $b_k, b_j, b_i \in \{0, 1\}^n$. Use the results to check that your implementation is correct. What do you observe about the auxiliary variables?

Task 4 (hard)

Using the Hamiltonian from Task 3 (twice), implement the problem Hamiltonian for the 2-bit ripple-carry adder circuit (cf. [https://en.wikipedia.org/wiki/Adder_\(electronics\)#Ripple-carry_adder](https://en.wikipedia.org/wiki/Adder_(electronics)#Ripple-carry_adder)).

Note that the full constraint involves 8 variables for each of the two full adders, with 1 carry variables shared across the adder pair, for a total of 15 variables. No auxiliary variables beyond the 6 used by the two full adders are required for this circuit.

Modify the Hamiltonian and use QAOA (multiple layers, if necessary) to find the solution $c \in \{0, \dots, 7\}$ to $c = 2 + 3$ (noting that $2 = 0b10$ and $3 = 0b11$).

Task 5 (hard)

Using the Hamiltonian from Task 2 (twice), together with the Hamiltonian for the and operation (four times), implement the problem Hamiltonian for the 2-bit Wallace multiplier circuit (cf. https://en.wikipedia.org/wiki/Wallace_tree).

Note that the full constraint involves 4 variables for each of the two half adders, plus 4 auxiliary variables for the bit products, for a total of 12 variables.

Modify the Hamiltonian and use QAOA (multiple layers, if necessary) to find solutions $x, y \in \{0, 1, 2, 3\}$ to $x \times y = 6$.

Task 6 (very hard)

Using the Hamiltonians from all previous tasks, implement the Hamiltonian corresponding to the 2-bit multiply-add circuit, computing the 4-bit result of $x + y \times z$ on 2-bit integers $x, y, z \in \{0, 1, 2, 3\}$.

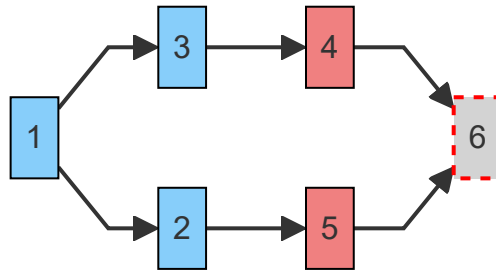
Modify the Hamiltonian and use QAOA (multiple layers, if necessary) to find solutions $x, y, z \in \{0, 1, 2, 3\}$ to $x + y \times z = 6$.

Submission

You should submit a single Jupyter notebook, with both code and results displayed. I should not have to re-run your notebook to see your work.

Assessment

The tasks in this assignment are designed to give you multiple opportunities to demonstrate appreciation and understanding of a variety of concepts and techniques in quantum computing. The tasks have the following dependency graph:



Note that you will not receive marks for a task if you've not attempted all the tasks it depends on. Different tasks have different levels of difficulty: blue for easy (1-3), red for hard (4-5), grey for very hard (6).

Below are the minimum requirements for each mark band:

- 2 easy tasks $\Rightarrow$ pass
- 3 easy tasks $\Rightarrow$ merit
- 2 easy tasks + 1 hard task $\Rightarrow$ distinction
- 3 easy tasks + 2 hard tasks $\Rightarrow$ model answer

Note that the maximum mark which can be achieved by completing tasks 1-5 is 85. The final 15 marks are reserved for submissions which go beyond my expectations, in the form of an attempt at task 6.